NLP: Text Classification

Kavyanjali Agnihotri
Matriculation Number: 6975801

1 Introduction

This report explains the steps and results of supervised text classification task. The goal was to predict the ontology class (among 14 classes) of a given text using Natural Language Processing techniques. The task uses 'dbpedia_14' dataset from HuggingFace and classifies the 'content' text field into 14 classes/labels.

2 Dataset and Preprocessing

2.1 Dataset

The dataset used is DBpedia 14, which contains text samples classified into 14 ontology classes. The original dataset has 630,000 entries (560,000 train and 70,000 test samples). To make training more manageable, the dataset was reduced while maintaining class balance, resulting in 98,000 training samples and 21,000 each for validation and testing. To maintain the class balance, the dataset was reduced by using

```
stratify=limited_df['label']
```

The reduced dataset column details were as follows:

```
{'label': ClassLabel(names=['Company',
'EducationalInstitution', 'Artist',
'Athlete', 'OfficeHolder',
'MeanOfTransportation', 'Building',
'NaturalPlace', 'Village', 'Animal',
'Plant', 'Album', 'Film',
'WrittenWork'], id=None),
'title': Value(dtype='string', id=None),
'content': Value(dtype='string', id=None)}
```

Key observations:

- The dataset is clean with no special characters or formatting issues
- Average word count per sample is 46.1 words
- Classes are perfectly balanced with 10,000 samples per class (in train set) in the reduced dataset, similarly for test and validation sets

Table 1: Basic Statistics Comparison Across Splits

Metric	Train	Validation	Test
Samples	98,000	21,000	21,000
Classes	14	14	14
Class Imbalance Ratio	1.00	1.00	1.00
Avg. length (words)	46.1	46.2	46.2
Median length (words)	46.0	46.0	46.0
Unique tokens	10,882	10,435	10,771

2.2 Cleaning and Preprocessing

The preprocessing steps included combining the original train and test set. Performing stratified sampling to maintain class balance.

Analysis of the reduced dataset is shown in table 1.

The top 5 unigrams in different splits are as: For 'train' split:

```
- school: 242
- district: 179
- film: 153
- born: 148
- album: 135
```

For 'test' split:

```
- school: 237
- district: 159
- born: 143
- film: 131
- album: 126
```

For 'validation' split:

```
- school: 184
- born: 144
- film: 143
- district: 140
- family: 134
```

The top bigram comparison between the splits is shown in the plot 1.

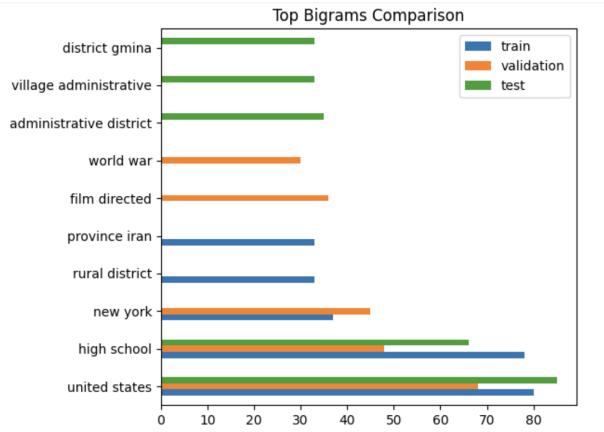


Figure 1: Plot showing the comparison between the top-bigrams in different splits

An interesting observation is that the content has barely any stopwords appearing in the top unigram and bigrams. Hence there was no need to perform extra cleaning as basic text cleaning was already done in the original dataset.

3 Dataset Preprocessing and Data Analysis

3.1 Tokenizer Selection

For the tokenization process, I used NLTK's `word_tokenize()` function, which implements the Treebank tokenizer. This regex-based tokenizer follows the Penn Treebank conventions and provides intelligent text segmentation. It has ability to handle contractions (e.g., converting "don't" to ["do", "n't"]) and properly manage punctuation. These characteristics make it particularly suitable for our English language processing tasks.

3.2 Text Cleaning Procedure

The text cleaning process involved several operations to prepare the data for analysis. First, convert all text to lowercase to ensure case-insensitive matching, which prevents the system from treating word variants with different capitalization as distinct tokens. Subsequently, special characters were removed using regular expressions to focus our analysis on the lexical content.

Then elimination of common English stopwords to reduce noise and decrease the dimensionality of our feature space. Finally, applying lemmatization to reduce words to their base forms, which provides a more semantic approach than stemming. This step helps group word variants (such as converting "running" to "run") while preserving the original meaning of the words.

Table 2: Basic Statistics Comparison

Metric	Train	Validation	Test
Samples	98,000	21,000	21,000
Classes	14	14	14
Class Imbalance	1.00	1.00	1.00
Avg. length	27.0	27.1	27.0
Median length	27.0	27.0	27.0
Unique tokens	9,112	8,839	9,081

3.3 Preprocessing Pipeline

The complete preprocessing pipeline consisted of five sequential stages: text normalization (combining lowercase conversion and character cleaning), tokenization, stopword filtering, lemmatization and token recombination. First, normalization creates a consistent textual foundation, then tokenization enables effective word-level analysis. Next, stopword removal focuses attention on content-bearing terms, and lemmatization reduces sparsity while maintaining semantic integrity. The final recombination step preserves document structure.

This comprehensive approach to preprocessing ensures that our textual data maintains its linguistic meaning while being optimally prepared for the next analysis phases of our project.

Analysis of the preprocessed text is as shown in table 2.

The top 5 unigrams in different splits are as: For 'train' split:

- `school: 259`
- `district: 184`
- `film: 161`
- `born: 144`
- `album: 143`

For 'test' split:

- `school: 251`
- `district: 159`
- `album: 143`
- `family: 140`
- `film: 137`

For 'validation' split:

- `school: 209`
- `family: 148`
- `film: 146`
- `album: 145`

- district: 137

The top bigram comparison between the preprocessed splits is shown in the plot 2.

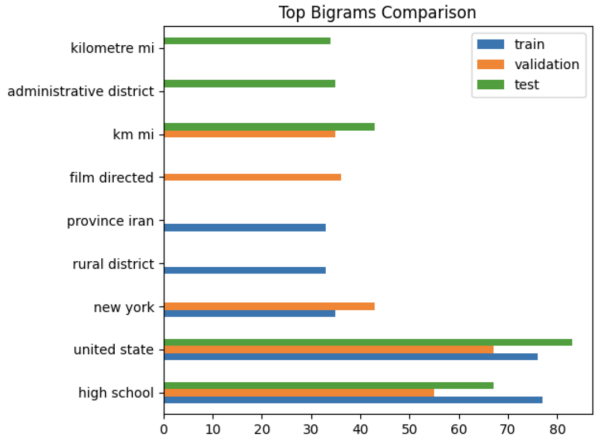


Figure 2: Plot showing the comparison between the top-bigrams in different preprocessed splits

There are two interesting observations. First, the average text length decreased significantly from about 46 to 27 because of stopword and special character removal. Second, the top unigrams and bigrams for the original and preprocessed text are almost same; this makes sense as the top unigram or bigrams in our original were not noisy with frequent stopwords. Few bigrams may have changed due to lemmatization.

4 Models

4.1 Naive Bayes with TF-IDF Features

The Naive Bayes classifier is a probabilistic model grounded in Bayes' Theorem, employing the feature independence assumption (the "naive" assumption) and positional independence through a bag-of-words approach. The core probability formulation is:

$$P(y|X) = \frac{P(y)\prod_{i=1}^{n} P(x_i|y)}{P(X)} \quad (1)$$

where $P(y)$ is the prior class probability, $P(x_i|y)$ the feature likelihood, and $P(y|X)$ the posterior probability we compute for classification.

4.1.1 Implementation Details

The implementation begins with data preparation, where we extract text content and labels from the preprocessed dataset splits.

We then apply TF-IDF vectorization with Scikit-learn's `TfidfVectorizer`, limiting features to the top 5,000 terms by frequency:

```
vectorizer = TfidfVectorizer(max_features=5000)
X_train_nb = vectorizer.fit_transform(train_texts)
X_val_nb = vectorizer.transform(val_texts)
```

The `MultinomialNB` implementation is particularly suited for TF-IDF transformed text data, as it models discrete feature counts:

```
nb_model = MultinomialNB()
nb_model.fit(X_train_nb, train_labels)
```

4.1.2 Technical Considerations

The TF-IDF transformation helps mitigate the impact of common stopwords while emphasizing discriminative terms. It does that by normalizing term frequency within documents and weighting inverse document frequency across the corpus.

Key advantages of this approach include computational / training efficiency of O(n), minimal memory requirements and natural handling of high-dimensional sparse data.

However, the model's strong independence assumptions limit its ability to capture word order and phrase patterns, contextual word relationships and complex semantic interactions.

This implementation is still effective for our task, achieving 94.3571% accuracy on our test set while requiring minimal computational resources compared to more complex models.

4.2 Neural Network Classifier

Our neural network architecture implements a feed-forward design specifically for text classification tasks. The model leverages modern deep learning techniques to capture complex linguistic patterns while remaining computationally efficient.

4.2.1 Data Preparation

The implementation begins with careful data preparation to ensure proper formatting and compatibility with our neural network architecture. All text content from the training set is converted to string format, while the corresponding labels are extracted separately. The system determines the number of unique classes from the training labels which allows for adaptation to different datasets without manual configuration. This automatic class detection ensures our model's output layer will have the appropriate dimensionality for the specific classification task at hand.

4.2.2 Text Vectorization

The text vectorization process transforms raw text into numerical representations suitable for neural network processing. Using TensorFlow's `TextVectorization` layer, a vocabulary size of 10,000 tokens to capture the most frequent words while controlling memory usage, and a fixed sequence length of 200 tokens to ensure consistent input dimensions. The vectorizer is configured to output integer indices (`output_mode='int'`). The vocabulary is learned from the training data through the `adapt()` method, which analyzes the text corpus to build the token-to-index mapping. This preprocessing step converts variable-length documents into fixed-length integer sequences that preserve the essential lexical information.

4.2.3 Model Architecture

The core architecture consists of the following layers:

Input $\rightarrow$ Vectorization $\rightarrow$ Embedding $\rightarrow$ Pooling $\rightarrow$ Dense $\rightarrow$ Output

The key design choices include a 64-dimensional embeddings to balance information density and computational efficiency, global average pooling for handling variable-length sequences, ReLU activation in the hidden layer for effective gradient flow and lastly softmax output for probabilistic multi-class classification.

4.2.4 Training Configuration and Process

The model is compiled with the below options:

```
nn_model.compile(
  # Adaptive learning rate
  optimizer='adam',
  # Suitable for integer labels
  loss='sparse_categorical_crossentropy',
  metrics=['accuracy']
)
```

The code utilizes batched TensorFlow datasets for efficient memory usage. The model undergoes training with validation monitoring and 100 epochs.

4.2.5 Technical Advantages

This architecture provides several benefits over traditional methods as it learns task-specific word embeddings during training. It handles variable-length documents through pooling and captures non-linear feature interactions. It can also be scaled efficiently to large datasets.

The model achieves 96.0333% test accuracy, demonstrating higher performance to our Naive Bayes while maintaining reasonable computational requirements.

5 Model Evaluation and Comparison

5.1 Performance Metrics Analysis

The random guessing accuracy achieved for our task was 7.23%, both models i.e. Naive Bayes and Neural Network trained for our task perform better than just random guessing. These models demonstrated strong performance on the classification task, with the Neural Network (NN) achieving slightly better overall accuracy (96.03%) compared to Naive Bayes (NB) (94.36%).

Complete metrics (Table 3) reveal model behavior. NB achieved Validation Accuracy of 94.30% and Test Accuracy of 94.36%, with macro and weighted precision, recall, and F1-score around 0.94, indicating consistent performance. NN achieved higher Test Accuracy of 96.03%, Validation Accuracy of 96.29%, and Training Accuracy of 99.76% (loss 0.0086), showing uniformly strong performance across all metrics and classes with macro and weighted averages of 0.96.

Table 3: Model Performance Comparison

Metric	Naive Bayes	Neural Network
Validation Accuracy	94.30%	96.29%
Test Accuracy	94.36%	96.03%
Macro Avg F1-score	0.94	0.96
Training Time	<1 min	100 epochs

The NN's superior performance is particularly evident in the F1-scores across most classes, suggesting better balance between precision and recall. However, NB's performance remains impressive considering its naive assumptions and computational efficiency.

5.2 Confusion Matrix Analysis

Figure 3 shows the confusion matrices for both models.

5.2.1 Naive Bayes

The Naive Bayes model performs well on most classes, particularly 3, 8, and 11, where F1-scores approach or exceed 0.98. However, it struggles with Classes 0, 2, 9, and 13. For example, Class 0 sees frequent confusion with Classes 11 and 13, and Class 2 with Classes 11 and 13 as well. Class 9

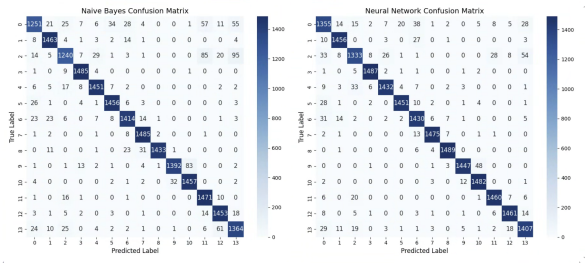


Figure 3: Confusion matrices for Naive Bayes (left) and Neural Network (right)

and 10 are also commonly misclassified with each other, indicating overlap in feature patterns.

Despite balanced support (1500 instances per class), Naive Bayes appears limited by its assumptions—particularly the conditional independence of features—which likely contributes to misclassifications where semantic or contextual similarity exists between classes.

5.2.2 Neural Network

The neural network significantly improves classification across the board. Class 0's recall rises from 0.83 to 0.91, and similar gains appear for Classes 2 and 13. Confusions between overlapping classes (e.g., 0 vs 11, 9 vs 10) are reduced, though not completely eliminated.

Its strength lies in modeling complex, non-linear feature relationships. While some confusion persists (e.g., Class 2 vs 13), the network's improved handling of semantic overlap and inter-feature dependencies results in higher precision and recall overall.

5.3 Strengths and Weaknesses

5.3.1 Naive Bayes

Strengths: Fast, efficient with small or high-dimensional data, and easy to implement.
Weaknesses: Struggles with contextual meaning and interdependent features; performance gains plateau early.

5.3.2 Neural Network

Strengths: Captures semantic and structural relationships; improves with more data; learns feature interactions.
Weaknesses: Requires more data, time, and compute; sensitive to hyperparameters.

5.4 Recommendations

Use **Naive Bayes** for fast, lightweight applications with small datasets. Use **Neural Networks** when accuracy is critical and resources permit. Given our balanced data and need for precision, the neural network is preferred, offering 1.7% higher accuracy. Still, Naive Bayes offers competitive performance with lower cost in constrained environments.

5.5 Error Analysis

Remaining errors largely stem from:

- **Semantic overlap**: e.g., creators vs. their works.
- **Ambiguity**: Some texts plausibly belong to multiple categories.
- **Low-frequency terms**: Rare words degrade model confidence.

Future efforts should explore more data, especially for ambiguous examples, or a hierarchical classification approach to reduce confusion between semantically close labels.